

Java Multithreading & Selenium/SDET

Senior SDET / Automation Interview Guide

Part 1: Core Java Multithreading **Part 2:** Multithreading in Selenium / SDET

PART 1 — CORE JAVA MULTITHREADING

1. What is Multithreading?

Multithreading means executing multiple threads concurrently within a process. A thread is a lightweight unit of execution. Multiple threads can make progress independently and can improve throughput for suitable workloads.

```
Main Thread
|
+---- Thread 1 → Task A
|
+---- Thread 2 → Task B
|
+---- Thread 3 → Task C
```

2. Creating a Thread using Thread

```
class MyThread extends Thread {

    @Override
    public void run() {

        System.out.println(
            Thread.currentThread().getName()
        );
    }
}

public class Test {

    public static void main(String[] args) {

        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.start();
        t2.start();
    }
}
```

Important: `start()` creates/schedules a new thread. Calling `run()` directly does not start a new thread; it is just a normal method call on the current thread.

3. Runnable

```
class MyTask implements Runnable {

    @Override
    public void run() {

        System.out.println(
            Thread.currentThread().getName()
        );
    }
}

Thread t1 = new Thread(new MyTask());
Thread t2 = new Thread(new MyTask());
```

```
t1.start();
t2.start();
```

Runnable is commonly preferred over extending Thread because the task is separated from the thread object, and the class remains free to extend another class.

4. sleep()

```
for (int i = 1; i <= 5; i++) {

    System.out.println(i);

    Thread.sleep(1000);
}
```

sleep() pauses the currently executing thread for approximately the requested duration. It throws InterruptedException and does not release intrinsic locks held by the thread.

5. join()

```
Thread t1 = new Thread(() -> {

    System.out.println("Task running");

});

t1.start();

t1.join();

System.out.println("Main continues");
```

join() makes the current thread wait for the target thread to terminate.

6. Race Condition

```
class Counter {

    int count = 0;

    void increment() {
        count++;
    }

}
```

When multiple threads update shared mutable state without proper coordination, updates can be lost. This is a race condition.

7. synchronized

```
class Counter {

    int count = 0;

    synchronized void increment() {

        count++;
    }

}
```

synchronized provides mutual exclusion for the synchronized monitor. For a synchronized instance method, the lock is associated with the object instance.

8. synchronized Block

```
void increment() {

    synchronized (this) {

        count++;
    }

}
```

```
}
```

A synchronized block can limit locking to the critical section instead of the whole method.

9. AtomicInteger

```
import java.util.concurrent.atomic.AtomicInteger;

AtomicInteger count = new AtomicInteger(0);

count.incrementAndGet();

System.out.println(count.get());
```

AtomicInteger is useful for atomic updates to a single integer value without using a traditional synchronized block for that operation.

10. ExecutorService

```
ExecutorService executor =
    Executors.newFixedThreadPool(3);

for (int i = 1; i <= 10; i++) {

    int taskNumber = i;

    executor.submit(() -> {

        System.out.println(
            "Task " + taskNumber +
            " : " +
            Thread.currentThread().getName()
        );
    });
}

executor.shutdown();
```

ExecutorService manages a pool of reusable worker threads. This is generally more practical than manually creating many Thread objects.

11. Callable and Future

```
ExecutorService executor =
    Executors.newFixedThreadPool(2);

Callable<Integer> task = () -> 10 + 20;

Future<Integer> future =
    executor.submit(task);

Integer result = future.get();

System.out.println(result);

executor.shutdown();
```

Runnable represents a task without a return value. Callable can return a value, and Future represents the pending result.

12. CompletableFuture

```
CompletableFuture
    .supplyAsync(() -> 10)
    .thenApply(n -> n * 2)
    .thenApply(n -> n + 5)
    .thenAccept(System.out::println);

// Output:
// 25
```

CompletableFuture supports asynchronous execution and composition of dependent stages.

13. ReentrantLock

```
private final ReentrantLock lock =
    new ReentrantLock();

lock.lock();

try {

    count++;

} finally {

    lock.unlock();

}
```

Always release a ReentrantLock in a finally block so an exception does not accidentally leave the lock held.

14. Concurrent Collections

```
ConcurrentHashMap<String, Integer> map =
    new ConcurrentHashMap<>();

map.put("Java", 1);
map.put("Selenium", 2);
```

Useful concurrent collections include ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueue and ConcurrentLinkedQueue.

PART 2 — MULTITHREADING IN SELENIUM / SDET

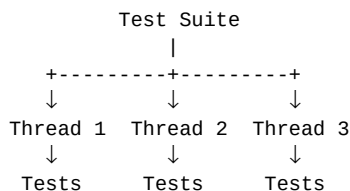
1. Why Parallel Test Execution?

A regression suite with hundreds or thousands of tests can take a long time when executed sequentially. Parallel execution allows independent tests to run concurrently, reducing elapsed execution time when the environment and tests support it.

Sequential

Test 1 → Test 2 → Test 3 → ... → Test 100

Parallel



2. Selenium WebDriver and Thread Safety

A WebDriver instance should not be casually shared between concurrently executing test threads. A safer parallel-test design gives each test thread its own driver instance.

Bad design: one static WebDriver shared by all test threads.

Better design: one WebDriver instance associated with each execution thread.

3. ThreadLocal ■

ThreadLocal provides each thread with its own independent value. It is a common pattern for managing WebDriver instances in parallel Selenium frameworks.

```
public class DriverManager {

    private static ThreadLocal<WebDriver> driver =
        new ThreadLocal<>();

    public static void setDriver(WebDriver webDriver) {

        driver.set(webDriver);
    }

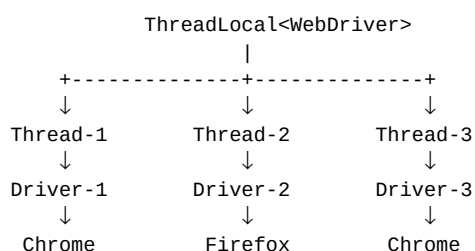
    public static WebDriver getDriver() {

        return driver.get();
    }

    public static void removeDriver() {

        driver.remove();
    }
}
```

4. ThreadLocal Architecture



5. Typical Driver Lifecycle

```

@BeforeMethod
↓
Create WebDriver
↓
driver.set(webDriver)
↓
Test executes
↓
@Before/After methods use getDriver()
↓
Quit WebDriver
↓
ThreadLocal.remove()

```

The exact annotations depend on the test framework. The key principle is to create, use, quit and remove the driver in a controlled lifecycle.

6. TestNG Parallel Execution

TestNG supports parallel execution through suite configuration. Common modes include methods, classes and tests.

```

<suite name="Regression" parallel="tests"
      thread-count="3">

    <test name="ChromeTests">
        ...
    </test>

    <test name="FirefoxTests">
        ...
    </test>

</suite>

```

The exact parallel mode should match the framework's isolation model. Tests must not depend on shared mutable state.

7. Selenium Grid

Selenium Grid can distribute browser sessions across machines or environments. It is useful when parallel execution also needs cross-browser or distributed execution.



8. Thread Safety in Test Data

Driver isolation alone is not enough. Parallel tests can still interfere if they share mutable test data, files, static variables, database records or other resources.

Check for: static mutable variables, shared files, common test users, shared database records, non-thread-safe utilities and shared report objects.

9. Thread-Safe Test Data Strategy

```

Thread 1 → User A → Test Data A → Driver A
Thread 2 → User B → Test Data B → Driver B
Thread 3 → User C → Test Data C → Driver C

```

Use isolated test data where possible, or coordinate access to shared resources when isolation is not possible.

10. Parallel API + UI Testing

In an automation framework, independent API and UI tests can often execute in parallel. The same thread-safety principles apply: avoid unsafe shared state and isolate resources.

11. Common SDET Interview Question

Question: How would you design a Selenium framework for parallel execution?

Answer: I would configure the test runner for parallel execution, avoid sharing a WebDriver instance across threads, maintain one driver per thread using ThreadLocal, and ensure test data and framework utilities are thread-safe. For cross-browser or distributed execution, I would use Selenium Grid or a cloud execution platform. I would also make sure driver cleanup happens reliably after each test.

12. Common Mistakes

- Using one static WebDriver instance for all parallel tests.
- Calling run() instead of start() when a new thread is intended.
- Modifying shared collections without considering thread safety.
- Using static mutable test data across parallel tests.
- Forgetting to quit WebDriver after the test.
- Forgetting ThreadLocal.remove() when using ThreadLocal in a long-lived thread pool.
- Assuming parallel execution is safe when tests depend on execution order.

13. Core Java vs Selenium/SDET — What to Remember

Core Java	Selenium / SDET
Thread	Parallel test worker
Runnable / Callable	Test task
ExecutorService	Controlled worker pool
synchronized / Lock	Protect shared framework resources
Atomic classes	Safe counters/state where appropriate
ConcurrentHashMap	Concurrent shared data structure
ThreadLocal	One driver/resource per thread
join / Future	Wait for asynchronous work

■ Senior SDET Revision Flow

```
Thread
↓
Runnable / Callable
↓
start() / join()
↓
Race Condition
↓
synchronized
↓
Atomic Classes
↓
ExecutorService
↓
Future / CompletableFuture
↓
Concurrent Collections
↓
ThreadLocal
↓
```

Parallel Test Execution



Selenium WebDriver Isolation



TestNG + Selenium Grid